



Facultad de Ingeniería.
Universidad de Concepción.

Sistema de Gestión de Torneos.

Desarrollo Orientado a Objeto.

Grupo 13 - Integrantes:

Javiera Aravena Gutiérrez 2025452715

Felipe de la Fuente 2025047624

Martín García Barro 2025425742

Introducción.

Un torneo, ya sea deportivo o de videojuegos, requiere de muchas organización, ya sea definir que formato se usara para competir, fechas, que se inscriban participantes, generar los enfrentamientos e ir actualizando los resultados y avances. Al hacer este trabajo de forma manual no seria extraño que terminen habiendo errores, de cálculos, fechas, entre otras cosas que pueden hacer que el evento carezca de orden.

El presente informe tiene como objetivo documentar el desarrollo de un Sistema de Gestión de Torneos, como una aplicación de escritorio desarrollada en lenguaje Java con la biblioteca gráfica Swing, buscando resolver los problemas que puedan surgir para el organizador del evento o sus participantes.

Desarrollo.

- **Análisis del problema:** El enunciado propone el desarrollo de un sistema capaz de facilitar la gestión de torneos deportivos o de videojuegos. Se indica que un organizador puede definir el nombre, la disciplina (fútbol, ajedrez, videojuegos, entre otras) y un formato principal de competencia (eliminatória directa, eliminatória doble, liga simple). Se entiende que el organizador puede inscribir participantes, jugadores individuales o equipos, almacenando sus nombres y datos de contacto o estos hacer la inscripción por su cuenta. A partir de los participantes inscritos y el formato elegido, el sistema debe generar automáticamente un calendario de enfrentamientos o un bracket inicial. Durante el desarrollo del torneo, cada resultado registrado debe actualizar de forma automática las posiciones, el avance del bracket o la tabla de clasificación y por su parte los usuarios deben poder visualizar en todo momento el estado del torneo, los próximos encuentros y las estadísticas generales. Del análisis surgieron diversas ideas, de como implementar y desarrollar el programa para seguir lo requerido y a su vez optimizar la experiencia del usuario, como por ejemplo que los datos se guarden y la diferenciación de roles entre el organizador de un torneo y los usuarios que solo participa, limitando que puede hacer cada uno.
- **Estrategias de solución:** Para resolver el problema se uso lo siguiente.
 - **Patrón Strategy:** El formato de un torneo se modeló mediante la interfaz `FormatoTorneo`, implementada por las clases `LigaSimple`, `EliminatóriaDirecta` y `EliminatóriaDoble`. La clase `Torneo` delega en el formato vigente tanto la generación de los enfrentamientos como la actualización de la clasificación, de modo que agregar un nuevo formato de competencia no exige modificar la clase `Torneo`. `PanelCrearTorneo` elige que instanciar según lo escogido por el usuario.
 - **Patrón Builder:** La creación de un torneo involucra varios datos obligatorios como su nombre o la disciplina. La clase `TorneoBuilder` funciona para ir construyendo el Torneo paso a paso, y al final, el método `build()` revisa que todos los datos este correcto para crear el torneo. `PanelCrearTorneo` lo utiliza directamente.
 - **Patrón Observer:** la clase `Torneo` mantiene una lista de observadores (interfaz `Observador`) y los notifica automáticamente cada vez que cambia su estado interno, ya sea al inscribirse un participante, generarse los partidos o registrarse un resultado. El `PanelBracket` se registra como observador y se redibuja solo cuando es estrictamente necesario.

- **Decisiones de diseño:** Además de los patrones mencionados, se tomaron distintas decisiones durante el desarrollo. Los cambios en la pantalla se resolvieron con un CardLayout, de forma que cada ventana se muestre u oculte sin necesidad de abrir ventanas adicionales. Nos percatamos de que sería necesario una base de datos local, para que los torneos y usuarios permanezcan en la computadora, como se escogió que cada torneo este asociado a su dueño y que puedan haber usuarios que quieran inscribirse sin necesidad de que el organizador lo haga por su cuenta además de que así, cada usuario que quiera participar pueda inscribirse una sola vez, entre otras cosas, por ende estos datos eran importantes de mantener para una experiencia más realista. Para identificar cada torneo se generó un código identificador de seis caracteres a partir de un UUID, pensado para que un organizador pueda compartirlo con las personas que deseen inscribirse. A su vez a pesar de que el participante se puede inscribir por cuenta propia también se mantuvo la opción de que el organizador pueda inscribir participantes por su cuenta.
- **Problemas encontrados y soluciones:** En el transcurso del desarrollo de la interfaz gráfica, teníamos el objetivo de incorporar una temática oscura con el fin de dar un aire más moderno al sistema y distanciarnos del diseño por defecto de Java Swing. Pero al intentar adaptar los menús desplegables de selección de opciones (JComboBox) y los botones de acción (JButton), nos encontramos con un problema de renderizado y contraste. Al oscurecer el fondo de estos componentes, el texto de las opciones se mantenía en su color oscuro predeterminado, camuflándose y volviéndose ilegible. Además, la lista de opciones de los menús desplegables no adoptaba los nuevos colores a menos que se reescribieran los renderizadores internos (ListCellRenderer), lo cual agregaba complejidad innecesaria. Con el fin de no retrasar el desarrollo y evitar un fallo grave de usabilidad donde el usuario no pudiera leer sus selecciones, tomamos la decisión de diseño de aplicar un estilo híbrido. Mantuvimos el tema oscuro para los contenedores principales, pero las casillas de selección y botones los dejamos con configuraciones de alto contraste y fondos más tradicionales para garantizar que cualquier persona pueda leer e interactuar con las opciones sin tener problemas de accesibilidad.

A su vez la lógica para avanzar de ronda en los formatos de eliminatoria (tanto simple como doble) requirió implementar un sistema de límites dinámicos para fragmentar la lista principal de partidos. La solución fue utilizar una lista de control (rondaLimites), lo cual permite al sistema identificar exactamente qué sub-lista de partidos pertenece cada ronda, facilitando la visualización correcta en el PanelBracket sin duplicar estructuras de datos innecesariamente.

A medida que se avanzaba con el desarrollo surgían diversos problemas a solucionar que no nos habíamos planteado desde un inicio, como por ejemplo, que el sistema tenga que identificar quien es el dueño del torneo, que pasa con los empates en las ligas que no lo permiten, que las fechas a colocar tengan coherencia, entre otros, finalmente fue cuestión de prueba y error, buscar la forma de que el resultado final tenga sentido y siga la idea general del proyecto.

Autocrítica.

Revisando el proceso de construcción del sistema, se identifican varios puntos que, de rehacerse el proyecto, se abordarían de otra manera.

- La separación entre modelo y vista se realizó como un refactor posterior en lugar de implementarlo desde el principio, lo que generó un trabajo que se hubiera evitado si se hiciera desde el inicio.
- Los test se desarrollaron al tener el trabajo terminado en vez de ir haciéndolo a la par se avanzaba en el desarrollo.
- El código de los distintos paneles repite decisiones de estilo visual en cada clase, en lugar de tenerlo en un único lugar, si luego se quieren volver a hacer cambios hay que cambiar los colores uno a uno.
- Había la intención de incorporar un creador de torneos libres, para que así el organizador puede crear un torneo sin limitaciones impuestas por las disciplinas escogidas, esto a medida que avanzamos en el proyecto quedó sin implementación y por ende finalmente descartado.
- Al querer implementar diversas cosas, hay pequeños detalles que quedaron sin configurar, se abarcó mucho pero a su vez eso impidió el poder optimizar y perfeccionar algunas funciones.

Propuesta de Mejoras:

A partir de la autocrítica anterior, se plantean las siguientes recomendaciones para proyectos futuros de características similares.

- Definir la arquitectura en capas desde la etapa de diseño, en lugar de reorganizarla de forma tardía.
- Hacer las pruebas de forma simultánea al desarrollo de la lógica del programa.
- Terminar y perfeccionar cada función propuesta antes de seguir agregando cosas.

Diagramas.

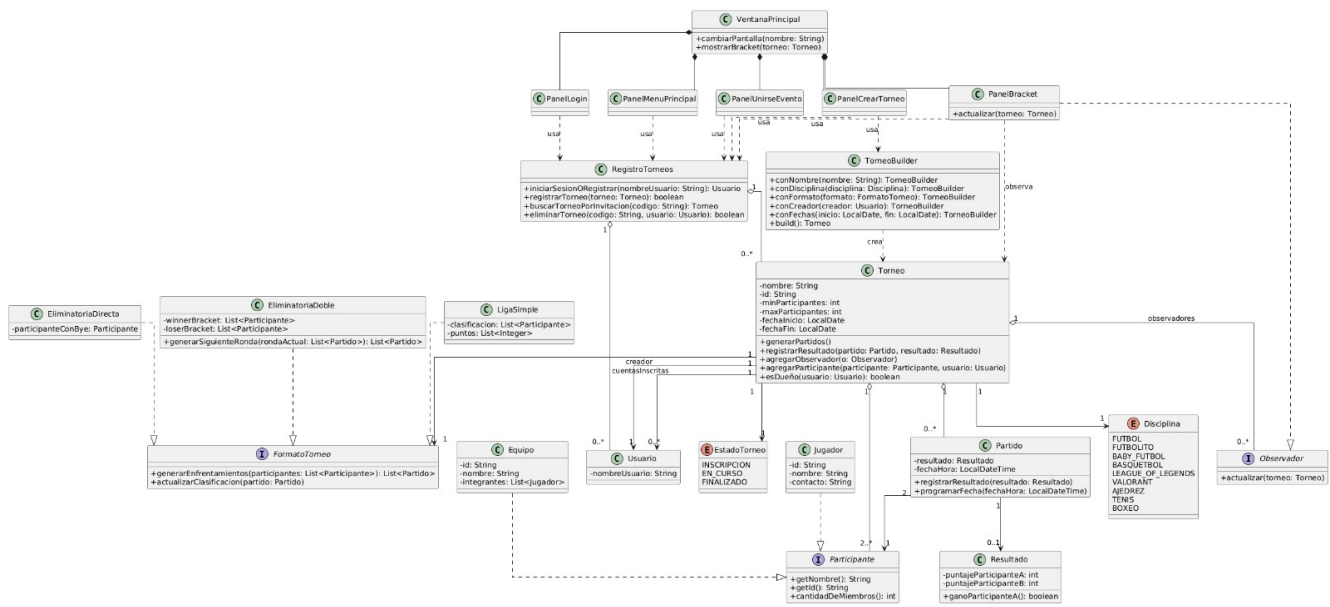


Figura 1: Diagrama de clases UML

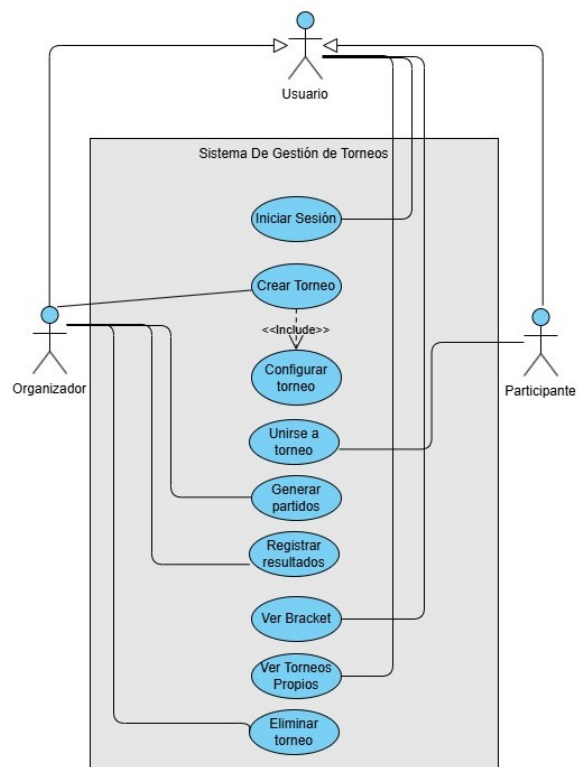


Figura 2: Diagrama de casos de uso.

Interfaz.

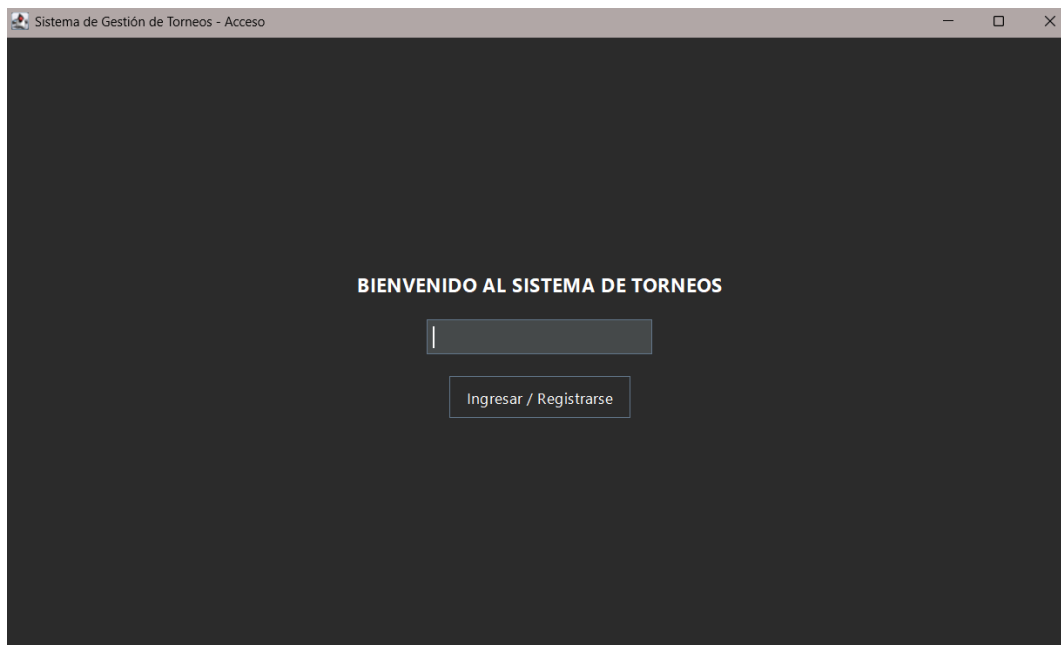


Figura 3: Panel de Login.



Figura 4: Panel de Menu Principal.

Sistema de Gestión de Torneos - Panel de Control

Nombre del Torneo: Mundial

Disciplina: FUTBOL

Formato: Eliminatória Directa

Mínimo de participantes: 32

Máximo de participantes: 32

Fecha Inicio (YYYY-MM-DD): 2026-07-09

Fecha Fin (YYYY-MM-DD): 2026-07-22

Volver al Menú

Crear y Guardar

Figura 5: Panel de Creación de torneo.

Sistema de Gestión de Torneos - Panel de Control

Código de Invitación (ID): 1F5F25

Buscar Torneo

Datos de Inscripción

Torneo encontrado: Mundial

Nombre del Equipo:

Integrantes (separados por saltos de línea):

Contactos (separados por salto de línea):

Inscribirse al Torneo

Volver al Menú

Figura 6: Panel de Unirse a Evento

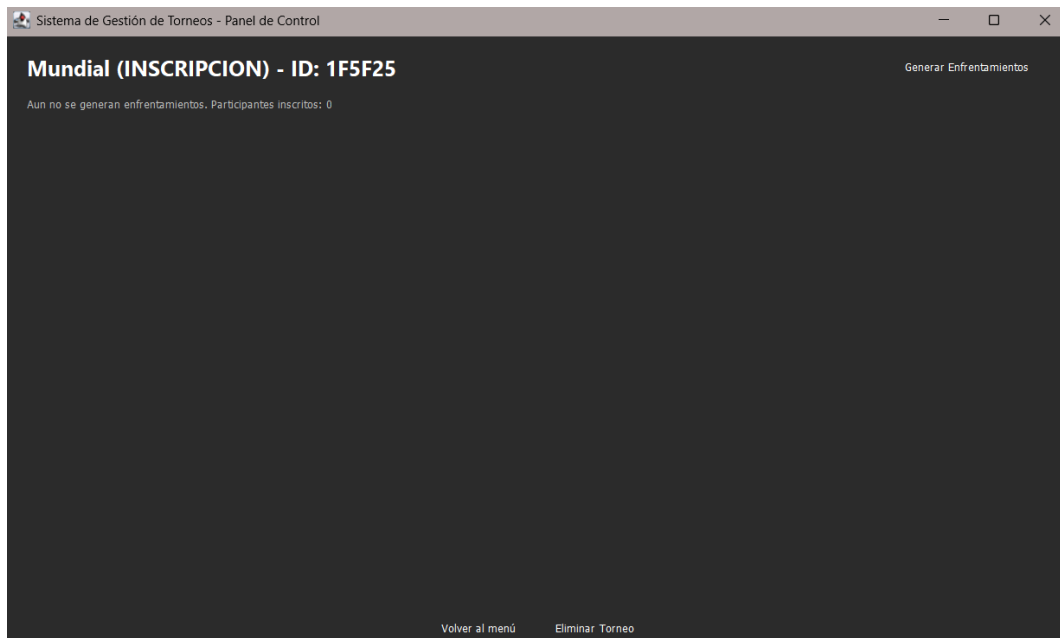


Figura 7: Panel de Bracket cuando el evento no ha comenzado.

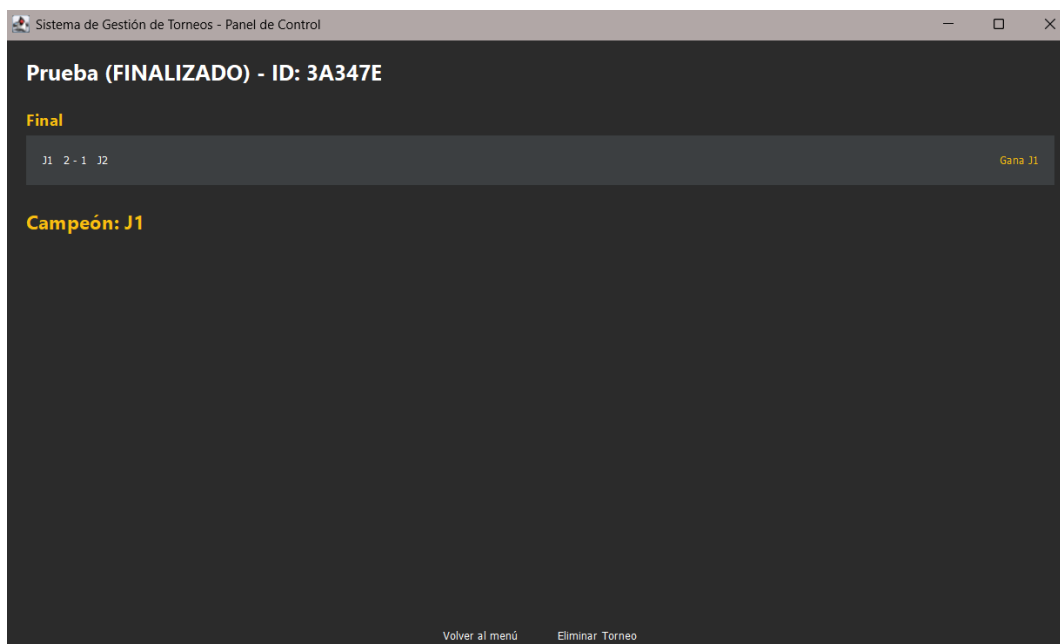


Figura 8: Panel de Bracket cuando el evento ya finalizó.

Conclusión.

El desarrollo del Sistema de Gestion de Torneos ha permitido consolidar los conocimientos adquiridos en torno al paradigma de la programación orientada a objetos. A través de la implementación de patrones de diseño como *Strategy*, *Builder* y *Observer*, se logró construir una aplicación rígida y capaz de adaptarse a diferentes formatos de competencia, y a su vez cumpliendo con los objetivos de funcionalidad, orden y estructura que fueron planteados inicialmente.

La experiencia de integrar una lógica compleja como la de un Gestor de Torneos resaltó la importancia de una planificación rigurosa y estructurada. Si bien la gestión de estados y persistencia de datos presentaron desafíos relativamente significativos, estas dificultades fueron fundamentales para comprender lo relevante que es la limpieza en el código y el desacoplamiento de las responsabilidades del sistema

Finalmente, este proyecto no solo permitió resolver la problemática de organizar eventos; tanto deportivos como de videojuegos, de forma automatizada; accesible al usuario y eficiente, sino que también funcionó como un ejercicio práctico de la aplicación de fundamentos esenciales a la hora de desarrollar un software. La importancia de las pruebas unitarias y la buena gestión de labores dentro del grupo servirán como base sólida para abordar desafíos más complejos en un futuro.